



Karachi Campus

Discovering Knowledge

Department Of Electrical Engineering

Formal out
gpt content
Deleted

Ref: 5/2

ROBOTIC SYSTEM & PROGRAMMING LAB

**Self Balancing Robot on ROS (Robotic Operating System)
using PID (Proportional Integral Derivatives) Controllers**

Group Members

Malahima Adeel	02-239232-043
Urooj Khan	02-239232-052
Shayan Ali	02-239232-068

BS(R&IS)-5A

FALL- 2025

Submitted to : Sir Hamza

10/Dec/25

Table of Contents

1. Abstract.....	1
2. Introduction	1
3. Literature Review	1
4. Components and Tools Description.	1
5. Methodology	2
5.1. ProposedModel.....	2
5.2. <u>Circuit/ Simulation Diagram.....</u>	2
5.3. <u>Circuit / Simulation Description.....</u>	2
6. Results and Discussions	3
7. Conclusion and Future Work	3
8. Project Summary	3
9. Project Pictures	4
10. References	4

1. ABSTRACT:

This project is about a Self-Balancing Robot using ROS (Robot Operating System) and Gazebo simulator. An open-source GitHub project was used as a base, and some extra files and changes were added according to project needs. During testing, some problems were found, such as extra cylinders appearing in the simulation and the GUI slider not resetting the robot properly. These problems are also discussed in this report.

2. INTRODUCTION:

A self-balancing robot is a robot that can stand upright on two wheels. It works like an inverted pendulum and needs continuous control to keep balance. If the robot tilts even a little, it must quickly correct itself, otherwise it will fall.

ROS is a popular platform used for developing robotic systems. It helps in communication between sensors, controllers, and actuators. Gazebo is used with ROS to simulate robots in a 3D environment. This allows testing without using real hardware. In this project, a self-balancing robot is simulated using ROS and Gazebo to understand its working and behavior.

3. LITERATURE REVIEW:

Many researchers have worked on self-balancing robots. Most of them use control techniques such as PID controllers to maintain balance. Research shows that accurate sensor data and proper control tuning are very important for stability.

Several ROS-based projects are available online that simulate self-balancing robots. These projects show that Gazebo is a powerful tool for testing robot models. However, studies also mention common problems such as model errors, duplicate parts, and controller reset issues.

4. COMPONENTS AND TOOLS DESCRIPTION:

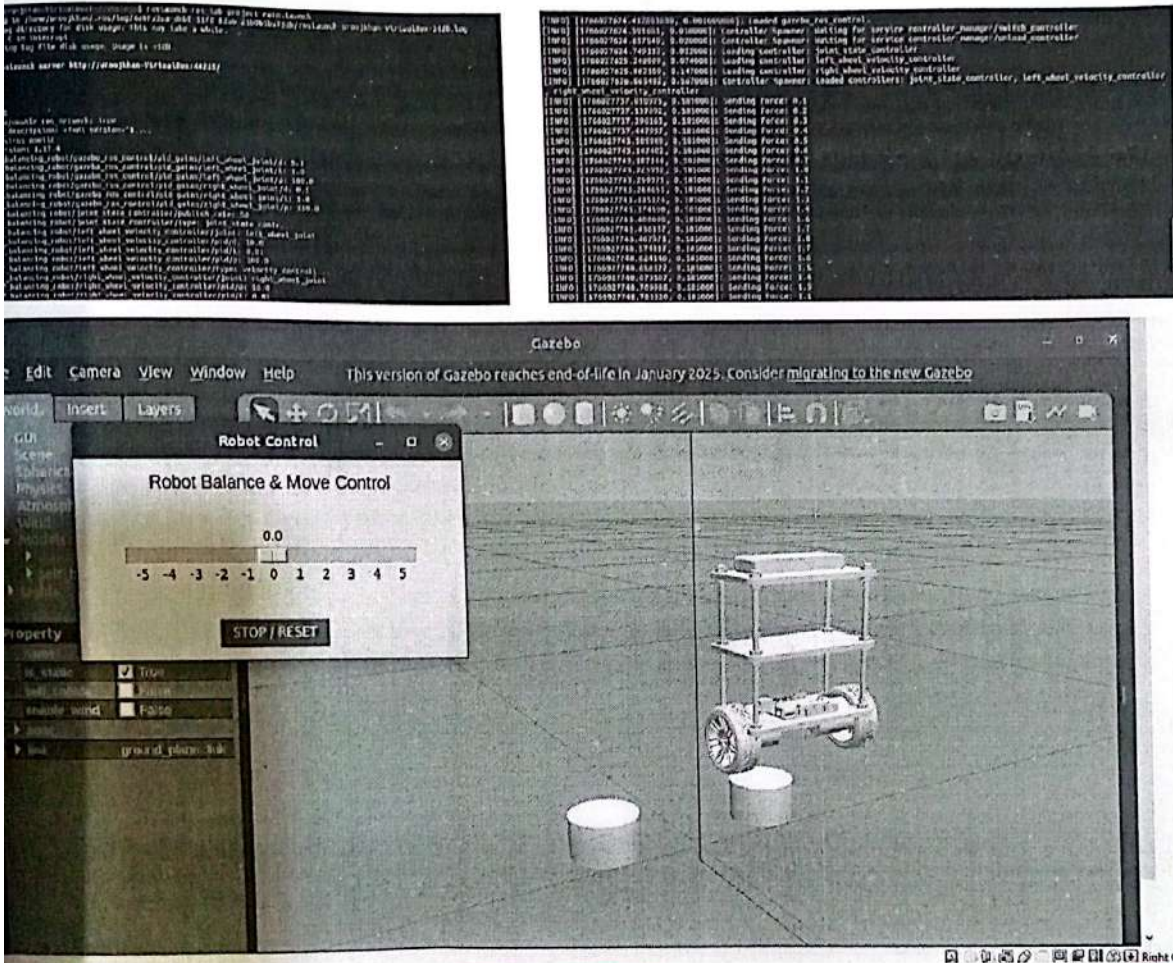
- **ROS:** Used for robot control and communication.
- **Gazebo:** Used for 3D robot simulation.
- **URDF/XACRO:** Used to design the robot model.
- **IMU Sensor:** Used to detect robot angle and movement.
- **PID Controller:** Used to balance the robot.

5. METHODOLOGY:

5.1. Proposed Model:

The proposed model is a two-wheeled self-balancing robot. The robot body is mounted above the wheels, and an IMU sensor is placed to measure tilt. The robot model is created using URDF files and launched in Gazebo using ROS launch files.

5.2. Simulation Diagram:



5.3. Circuit / Simulation Description:

When the simulation starts, the robot is loaded into Gazebo. The IMU sensor continuously measures the robot's angle. This data is sent to the controller node. The controller then calculates wheel movements to keep the robot upright. A GUI slider is provided to start, stop, and reset the robot. Ideally, the robot should return to its original position when reset is pressed, but this did not work correctly.

6. RESULTS AND DISCUSSIONS:

The robot was successfully launched in Gazebo and showed balancing behavior. The controller responded to small movement..

However, two major problems were observed:

1. Extra Cylinders Appearing:

Two extra cylinder shapes appeared along with the robot. This problem is likely due to duplicate link or collision definitions in the URDF or XACRO files. These extra objects affected the simulation and caused confusion.

2. GUI Slider Reset Issue:

When the stop or reset button was pressed, the robot did not return to its original upright position. The robot is continuously moving. This issue may be caused by improper controller reset, missing reset commands, or simulation state not being reset correctly.

7. CONCLUSION AND FUTURE WORK:

Conclusion:

This project successfully demonstrated a self-balancing robot in a simulated environment using ROS and Gazebo. The robot model and controller helped in understanding balancing behavior. Although some issues were faced, the project achieved its learning goals.

Future Work:

In the future, the extra cylinders can be removed by fixing URDF files. The GUI slider reset function can be improved by properly resetting the controller and simulation states. More advanced controllers can also be implemented. This project can later be implemented on real hardware.

8. PROJECT SUMMARY:

This project focused on the simulation of a self-balancing robot using ROS. The robot was modeled, controlled, and tested in Gazebo. Problems faced during simulation were identified and discussed. Overall, this project improved understanding of robotic simulation and control systems.

9. PROJECT PICTURES:

[illegible]

```

1  package com.hereon.hereon.issues.issues_0
2
3  package object issues_0 {
4
5    // via package0.mt
6    def package0_m0: Int = 0
7
8    // via package0.mt
9    def package0_m1: Int = 1
10
11    // via package0.mt
12    def package0_m2: Int = 2
13
14    // via package0.mt
15    def package0_m3: Int = 3
16
17    // via package0.mt
18    def package0_m4: Int = 4
19
20    // via package0.mt
21    def package0_m5: Int = 5
22
23    // via package0.mt
24    def package0_m6: Int = 6
25
26    // via package0.mt
27    def package0_m7: Int = 7
28
29    // via package0.mt
30    def package0_m8: Int = 8
31
32    // via package0.mt
33    def package0_m9: Int = 9
34
35    // via package0.mt
36    def package0_m10: Int = 10
37
38    // via package0.mt
39    def package0_m11: Int = 11
40
41    // via package0.mt
42    def package0_m12: Int = 12
43
44    // via package0.mt
45    def package0_m13: Int = 13
46
47    // via package0.mt
48    def package0_m14: Int = 14
49
50    // via package0.mt
51    def package0_m15: Int = 15
52
53    // via package0.mt
54    def package0_m16: Int = 16
55
56    // via package0.mt
57    def package0_m17: Int = 17
58
59    // via package0.mt
60    def package0_m18: Int = 18
61
62    // via package0.mt
63    def package0_m19: Int = 19
64
65    // via package0.mt
66    def package0_m20: Int = 20
67
68    // via package0.mt
69    def package0_m21: Int = 21
70
71    // via package0.mt
72    def package0_m22: Int = 22
73
74    // via package0.mt
75    def package0_m23: Int = 23
76
77    // via package0.mt
78    def package0_m24: Int = 24
79
80    // via package0.mt
81    def package0_m25: Int = 25
82
83    // via package0.mt
84    def package0_m26: Int = 26
85
86    // via package0.mt
87    def package0_m27: Int = 27
88
89    // via package0.mt
90    def package0_m28: Int = 28
91
92    // via package0.mt
93    def package0_m29: Int = 29
94
95    // via package0.mt
96    def package0_m30: Int = 30
97
98    // via package0.mt
99    def package0_m31: Int = 31
100
101    // via package0.mt
102    def package0_m32: Int = 32
103
104    // via package0.mt
105    def package0_m33: Int = 33
106
107    // via package0.mt
108    def package0_m34: Int = 34
109
110    // via package0.mt
111    def package0_m35: Int = 35
112
113    // via package0.mt
114    def package0_m36: Int = 36
115
116    // via package0.mt
117    def package0_m37: Int = 37
118
119    // via package0.mt
120    def package0_m38: Int = 38
121
122    // via package0.mt
123    def package0_m39: Int = 39
124
125    // via package0.mt
126    def package0_m40: Int = 40
127
128    // via package0.mt
129    def package0_m41: Int = 41
130
131    // via package0.mt
132    def package0_m42: Int = 42
133
134    // via package0.mt
135    def package0_m43: Int = 43
136
137    // via package0.mt
138    def package0_m44: Int = 44
139
140    // via package0.mt
141    def package0_m45: Int = 45
142
143    // via package0.mt
144    def package0_m46: Int = 46
145
146    // via package0.mt
147    def package0_m47: Int = 47
148
149    // via package0.mt
150    def package0_m48: Int = 48
151
152    // via package0.mt
153    def package0_m49: Int = 49
154
155    // via package0.mt
156    def package0_m50: Int = 50
157
158    // via package0.mt
159    def package0_m51: Int = 51
160
161    // via package0.mt
162    def package0_m52: Int = 52
163
164    // via package0.mt
165    def package0_m53: Int = 53
166
167    // via package0.mt
168    def package0_m54: Int = 54
169
170    // via package0.mt
171    def package0_m55: Int = 55
172
173    // via package0.mt
174    def package0_m56: Int = 56
175
176    // via package0.mt
177    def package0_m57: Int = 57
178
179    // via package0.mt
180    def package0_m58: Int = 58
181
182    // via package0.mt
183    def package0_m59: Int = 59
184
185    // via package0.mt
186    def package0_m60: Int = 60
187
188    // via package0.mt
189    def package0_m61: Int = 61
190
191    // via package0.mt
192    def package0_m62: Int = 62
193
194    // via package0.mt
195    def package0_m63: Int = 63
196
197    // via package0.mt
198    def package0_m64: Int = 64
199
200    // via package0.mt
201    def package0_m65: Int = 65
202
203    // via package0.mt
204    def package0_m66: Int = 66
205
206    // via package0.mt
207    def package0_m67: Int = 67
208
209    // via package0.mt
210    def package0_m68: Int = 68
211
212    // via package0.mt
213    def package0_m69: Int = 69
214
215    // via package0.mt
216    def package0_m70: Int = 70
217
218    // via package0.mt
219    def package0_m71: Int = 71
220
221    // via package0.mt
222    def package0_m72: Int = 72
223
224    // via package0.mt
225    def package0_m73: Int = 73
226
227    // via package0.mt
228    def package0_m74: Int = 74
229
230    // via package0.mt
231    def package0_m75: Int = 75
232
233    // via package0.mt
234    def package0_m76: Int = 76
235
236    // via package0.mt
237    def package0_m77: Int = 77
238
239    // via package0.mt
240    def package0_m78: Int = 78
241
242    // via package0.mt
243    def package0_m79: Int = 79
244
245    // via package0.mt
246    def package0_m80: Int = 80
247
248    // via package0.mt
249    def package0_m81: Int = 81
250
251    // via package0.mt
252    def package0_m82: Int = 82
253
254    // via package0.mt
255    def package0_m83: Int = 83
256
257    // via package0.mt
258    def package0_m84: Int = 84
259
260    // via package0.mt
261    def package0_m85: Int = 85
262
263    // via package0.mt
264    def package0_m86: Int = 86
265
266    // via package0.mt
267    def package0_m87: Int = 87
268
269    // via package0.mt
270    def package0_m88: Int = 88
271
272    // via package0.mt
273    def package0_m89: Int = 89
274
275    // via package0.mt
276    def package0_m90: Int = 90
277
278    // via package0.mt
279    def package0_m91: Int = 91
280
281    // via package0.mt
282    def package0_m92: Int = 92
283
284    // via package0.mt
285    def package0_m93: Int = 93
286
287    // via package0.mt
288    def package0_m94: Int = 94
289
290    // via package0.mt
291    def package0_m95: Int = 95
292
293    // via package0.mt
294    def package0_m96: Int = 96
295
296    // via package0.mt
297    def package0_m97: Int = 97
298
299    // via package0.mt
300    def package0_m98: Int = 98
301
302    // via package0.mt
303    def package0_m99: Int = 99
304
305    // via package0.mt
306    def package0_m100: Int = 100
307
308    // via package0.mt
309    def package0_m101: Int = 101
310
311    // via package0.mt
312    def package0_m102: Int = 102
313
314    // via package0.mt
315    def package0_m103: Int = 103
316
317    // via package0.mt
318    def package0_m104: Int = 104
319
320    // via package0.mt
321    def package0_m105: Int = 105
322
323    // via package0.mt
324    def package0_m106: Int = 106
325
326    // via package0.mt
327    def package0_m107: Int = 107
328
329    // via package0.mt
330    def package0_m108: Int = 108
331
332    // via package0.mt
333    def package0_m109: Int = 109
334
335    // via package0.mt
336    def package0_m110: Int = 110
337
338    // via package0.mt
339    def package0_m111: Int = 111
340
341    // via package0.mt
342    def package0_m112: Int = 112
343
344    // via package0.mt
345    def package0_m113: Int = 113
346
347    // via package0.mt
348    def package0_m114: Int = 114
349
350    // via package0.mt
351    def package0_m115: Int = 115
352
353    // via package0.mt
354    def package0_m116: Int = 116
355
356    // via package0.mt
357    def package0_m117: Int = 117
358
359    // via package0.mt
360    def package0_m118: Int = 118
361
362    // via package0.mt
363    def package0_m119: Int = 119
364
365    // via package0.mt
366    def package0_m120: Int = 120
367
368    // via package0.mt
369    def package0_m121: Int = 121
370
371    // via package0.mt
372    def package0_m122: Int = 122
373
374    // via package0.mt
375    def package0_m123: Int = 123
376
377    // via package0.mt
378    def package0_m124: Int = 124
379
380    // via package0.mt
381    def package0_m125: Int = 125
382
383    // via package0.mt
384    def package0_m126: Int = 126
385
386    // via package0.mt
387    def package0_m127: Int = 127
388
389    // via package0.mt
390    def package0_m128: Int = 128
3
```

[illegible]

```

100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

```

[illegible]

10. REFERENCES:

- Self-Balancing Robot GitHub Project:
https://github.com/sezan92/self_balancing_robot

ADDITIONAL INFORMATION:

CURL: Not used in this project because we used apt packages, git clone, and ROS tools.

1. Apt packages: Ubuntu's package manager that installs software from online repositories. We used `sudo apt-get install` to get ROS controller packages.

2. Git clone: Copies a GitHub repository to local computer.
In our project we cloned `self_balancing_robot` from GitHub to get URDF files.

3. ROS Tools: ROS-specific commands for building and running robotics projects.
We have used `catkin_make` to build and `roslaunch` to run simulations.

Course name :ROBOTIC SYSTEM AND PROGRAMMING
Course code:RIL-361

Table 3: Range of Complex Engineering Activities

Sr.	Attributes	Complex Activities
1.	Preamble	This project involves designing and simulating a self-balancing robot using robotics principles.
2.	Range of Resources	The project uses ROS, Gazebo, robot models, open-source code, and robotics tools.
3.	Level of Interaction	Sensors, controllers, and robot parts interact to maintain balance and control motion.
4.	Innovation	An existing robotics project was modified and tested with custom simulation changes.
5.	Consequences to society	Self-balancing robots support future robotic mobility and service applications.
6.	Familiarity	The project applies robotics and control concepts we did in lab like urdf modelling.